

Universal Hash Functions

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**



Choosing a Good Hash Function In Practice

- We want $h()$ to behave somewhat “randomly”.
 - Good Examples:
 - $h(k) = k \bmod m$.
 - $h(k) = \lfloor mk / C \rfloor$.
 - $h(k) = (ak) \bmod m$.
 - $h(k) = (ak + b) \bmod m$.
- m = table size
 C = max possible
key value
- } a and b should be chosen in a somewhat
“arbitrary” fashion (primes often used).
- Be sure to fully utilize “all the bits” in a key. For example, $h(k) = k \bmod 256$ is a somewhat poor hash function if k is a 32-bit IP address.
 - Be sure to use the entire hash table; e.g., make sure $h(k)$ isn’t always even.

So Far...

Insert	Remove	Find
$O(n)$	$O(n)$	$O(\log n)$
$O(1)$	$O(1)$, post-find	$O(n)$
$O(1)$, post-find	$O(1)$, post-find	$O(n)$
$O(1)$	$O(1)$, post-find	$O(n)$
$O(\log n)$	$O(\log n)$	$O(\log n)$
$O(1)$ amortized	$O(1)$ amortized	$O(1)$, we hope

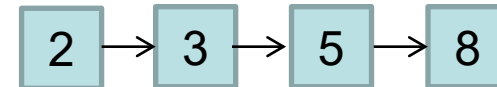
Sorted array



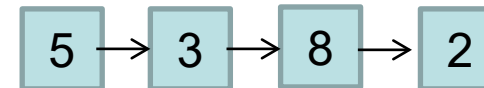
Unsorted array



Sorted (doubly) linked list



Unsorted (doubly) linked list

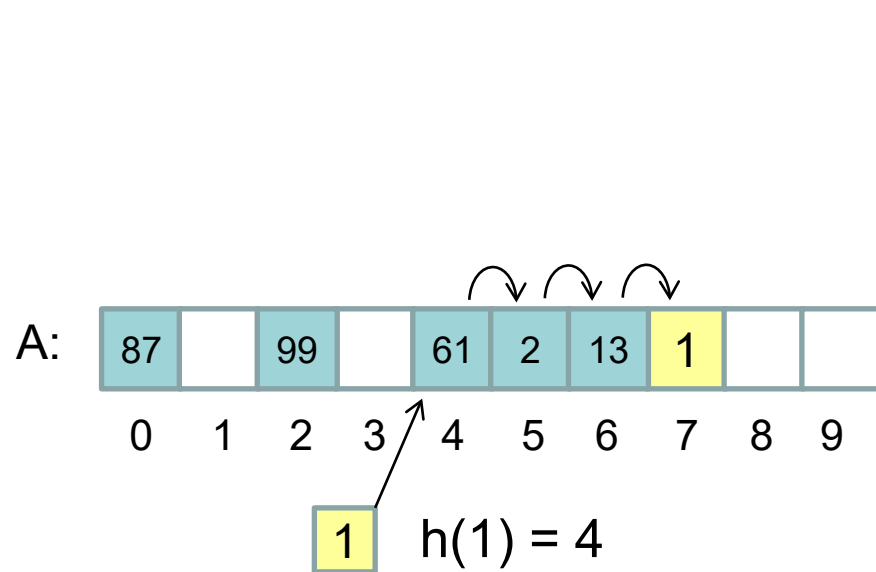


Balanced BST, skip list, B-tree

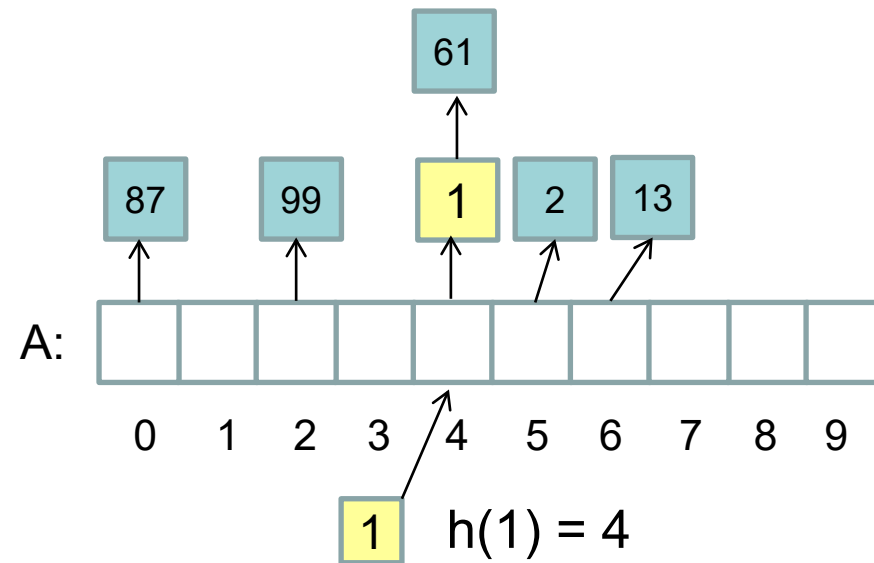
Hash tables with heuristic choices for hash functions

Collision Resolution

- Probing: *Insert* and *find* same runtime; *delete* $O(1)$.
 - Chaining: *Insert* and *delete* $O(1)$; *find* interesting.
- (note: we often separate the functionality of *delete* and *find*)



(Sequential) Probing



Chaining

The Case Against Deterministic Hash Functions

- Any deterministic hash function has a “bad” set of input keys, causing *find* to take $\Theta(n)$ time.
- So if we want a good worst-case guarantee for *find*, we cannot use a deterministic hash function. We **must** use randomness in some way.

Why Not Use a Completely Random Hash Function?

- Suppose we choose a hash function $h(k)$ that maps every key $k \in \{0, \dots, C - 1\}$ to a completely random location in $\{0, \dots, m - 1\}$.
- For a chained hash table, $\mathbf{E}[\text{length of list we search}] = n / m$, so *find* does indeed run in $O(1 + n / m)$ time.
 - This is just simple “balls in bins” analysis.
 - Assume we use dynamic resizing to maintain $m = \Theta(n)$, so $n / m = O(1)$.
- With sequential probing, $\mathbf{E}[\text{running time for insert / find}] = O(1)$ also, as long as m is at least a constant factor larger than n .
- Fatal flaw: requires $\Theta(C)$ space to store the hash function, same as the direct access table!

Universal Hashing

- “Partially-random” – with a small # of random parameters chosen during initialization.
- A hash function is:
 - *Universal*, if $\Pr[\text{different keys } k_1 \text{ \& } k_2 \text{ colliding}] = 1 / m. \quad (O(1 / m))$
 - *Strongly universal*, if for any two (different) keys k_1 and k_2 and any two table cells c_1 and c_2 we have $\Pr[h(k_1) = c_1 \text{ and } h(k_2) = c_2] = 1 / m^2. \quad (O(1 / m^2))$
(strongly universal implies universal)
- Strongly universal means mappings are pairwise independent.
- Key punchline: $\mathbf{E}[\text{runtime for } \textit{find} \text{ in chained hash table}] = O(1 + n / m)$
if universal! (so $O(1)$ as long as $m = \Theta(n)$)

Universal Hashing – Examples

- **Strongly universal (hence also universal):**

$$h(k) = [(ak + b) \bmod p] \bmod m, \text{ with } \begin{cases} p: \text{any prime number } \geq C. \\ a: \text{random integer in } \{1, \dots, p-1\}. \\ b: \text{random integer in } \{0, \dots, p\}. \end{cases}$$

$$h(k) = \lfloor [(ak + b) \bmod (mC)] / C \rfloor, \text{ with } \begin{cases} a: \text{random integer in } \{0, \dots, mC-1\}. \\ b: \text{random integer in } \{0, \dots, mC-1\}. \end{cases}$$

$$h(k) = H_1[k_1] \text{ xor } H_2[k_2] \text{ xor } \dots \text{ xor } H_b[k_b], \text{ with } \begin{cases} m \text{ and } C \text{ being powers of } 2 \\ k_1 \dots k_b : \text{equal-sized blocks of bits into} \\ \text{which we have divided } k \text{ in binary} \\ H_1 \dots H_b : \text{fully-random hash functions} \\ \text{mapping to } \{0, \dots, m-1\} \end{cases}$$

- **Universal but not strongly universal:**

$$h(k) = \lfloor amk / C \rfloor \bmod m, \text{ with } \begin{cases} m \text{ and } C \text{ being powers of } 2 \\ a: \text{random odd integer in } \{1, 3, 5, \dots, C-1\} \end{cases}$$

$O(1)$ Expected Running Time for Find With Universal Hashing

- Consider a chained hash table.
- L : length of list we search in an unsuccessful call to $find(k)$.
(note: $\mathbf{E}[L] \geq \mathbf{E}[\text{length of list searched for a successful call to } find]$)
- We can easily compute $\mathbf{E}[L]$ using linearity of expectation...
- Let $k_1 \dots k_n$ be the keys stored in our table.
- Write $L = X_1 + X_2 + \dots + X_n$ where $X_j = 1$ if $h(k_j) = h(k)$.
- Since $k \neq k_j$, $\mathbf{E}[X_j] = O(1 / m)$ using universal hashing.
- So $\mathbf{E}[L] = \mathbf{E}[X_1] + \dots + \mathbf{E}[X_n] = n \mathbf{E}[X_j] = O(n / m) = O(1)$.

Summary: Chained Universal Hash Tables

- **Pr**[two different keys colliding] = $O(1/m)$.
- *Insert* and *delete*: $O(1)$ amortized time
- *Find*: $O(1)$ expected time

Example Applications

- Consider the following problems:
 - **Element Uniqueness:** Given n numbers $A_1 \dots A_n$, are all of these distinct, or are two of them equal?
 - **Set Intersection / Union / Difference:** Given two sets A and B (specified by unsorted arrays) containing n elements in total, output an array containing $A \cap B$, $A \cup B$, or $A \setminus B$.
- All of these have $\Omega(n \log n)$ worst-case lower bounds in the comparison model.
- But all can be solved in $O(n)$ expected time with universal hashing!
- Example problem (from our coding discussion): remove duplicates from a list of n items. $\leq n \text{ inserts } (O(n)) + n \text{ finds } (O(n) \text{ expected})$

Levels of Independence and Analysis For Probing-Based Tables

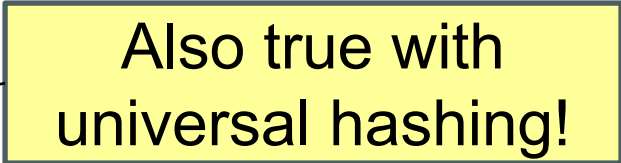
- Strongly universal: provides pairwise independence.
 - If we only look at 2 keys at a time, these are hashed in a completely random fashion, just as with a fully random function.
 - E.g., “linear” hashing: $h(k) = [(ak + b) \bmod p] \bmod m$
- Higher levels of independence are possible using higher-degree polynomial hash functions. For example,
$$h(k) = [(ak^3 + bk^2 + ck + d) \bmod p] \bmod m$$
is 4-wise independent.
- [Pagh et al. '11, Patrascu-Thorup '10]: 5-wise independence necessary and sufficient for $O(1)$ expected performance of *insert* / *find* with sequential probing.

Recall: Expected Collisions and the Birthday Paradox

- Throw N balls randomly into M bins.
 - $\mathbf{E}[\text{number of pairwise collisions}] = N(N - 1) / (2M) \approx N^2 / (2M)$
 - Number of pairs: $\binom{N}{2} = N(N - 1) / 2$
 - Probability a pair collides: $1 / M$
- $\mathbf{E}[\text{total collisions}] = \sum_{i < j} \mathbf{E}[X_{ij}]$ (where $X_{ij} = 1$ if balls i & j collide)

Remember we set $M = 365$ to analyze the “birthday paradox”...

Recall: Expected Collisions and the Birthday Paradox

- Throw N balls randomly into M bins.
 - $\mathbf{E}[\text{number of pairwise collisions}] = N(N - 1) / (2M) \approx N^2 / (2M)$
 - Number of pairs: $\binom{N}{2} = N(N - 1) / 2$
 - Probability a pair collides: $1 / M$ 
- $\mathbf{E}[\text{total collisions}] = \sum_{i < j} \mathbf{E}[X_{ij}]$ (where $X_{ij} = 1$ if balls i & j collide)

If $M = N^2$, then $\mathbf{E}[\# \text{ collisions}] \leq 1/2$, so $\mathbf{Pr}[\# \text{ collisions} \geq 1] \leq 1/2$ (by Markov), so $\mathbf{Pr}[\text{collision free}] \geq 1/2$ and hence $\mathbf{E}[\text{rebuild attempts until collision free}] \leq 2 \rightarrow$ Can build (static) collision-free hash table with $O(N^2)$ expected time and space.

Static “Perfect” Hashing ($O(1)$ Running time for Find)

- Can build a collision-free (static) hash table on n elements with $O(n^2)$ expected time and space.
- Resolve collisions in a top-level size- n universal table with these collision-free tables.
- If cell i has B_i elements,
 $\mathbf{E}[\text{build time}] = n + \mathbf{E}[\sum B_i^2]$
- With *exactly* the same math as bucket sort, this is $O(n)$.

